

PAL: Program-aided Language Models

Preliminary structure for a 30-minute seminar presentation

Eric Peter

Heidelberg University

Preliminary discussion

Part	Goal	Time
1. Motivation and background	Why reasoning with LLMs is hard; few-shot and chain-of-thought prompting	5 min
2. Core idea of PAL	Explain how programs become intermediate reasoning steps and why the interpreter matters	8 min
3. Experimental setup	Tasks, baselines, models, and evaluation protocol	5 min
4. Results and analysis	Main quantitative results, robustness, ablations, and failure modes	8 min
5. Discussion and conclusion	Strengths, limitations, and seminar discussion questions	4 min

Main message

PAL separates **understanding/decomposition** from **execution**: the LLM writes code-like reasoning steps, while Python performs the exact computation.

Motivation: LLMs reason, but they also calculate unreliably

Observation behind PAL

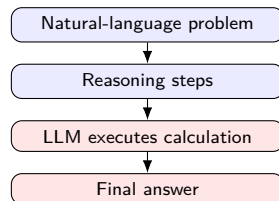
Large language models can often translate a natural-language problem into useful intermediate steps, but may still make arithmetic or logical execution mistakes.

- Few-shot prompting shows examples at test time.
- Chain-of-thought prompting asks the model to write intermediate reasoning steps.
- But in CoT, the same model both *plans* and *executes* the solution.

Source: PAL paper, Abstract and Introduction.

Problem

Even if the decomposition is plausible, the final answer can be wrong because the LLM performs calculations as text generation rather than exact computation.



Background 1: few-shot prompting

Definition

Few-shot prompting gives the LLM a small number of demonstrations directly in the input prompt. The model is not fine-tuned; it completes the new test instance by following the demonstrated pattern.

$$p = \langle x_1 \cdot y_1 \rangle \parallel \langle x_2 \cdot y_2 \rangle \parallel \cdots \parallel \langle x_k \cdot y_k \rangle$$

$$\text{Inference input: } p \parallel x_{\text{test}} \Rightarrow y_{\text{test}}$$

Intuition

The prompt teaches the model the expected format and the task behavior from examples instead of changing model parameters.

Source: PAL paper, Section 2.

Background 2: chain-of-thought prompting

Extension of few-shot prompting

Chain-of-thought adds intermediate natural-language reasoning steps t_i to each demonstration.

$$p = \langle x_1 \cdot t_1 \cdot y_1 \rangle \parallel \langle x_2 \cdot t_2 \cdot y_2 \rangle \parallel \cdots \parallel \langle x_k \cdot t_k \cdot y_k \rangle$$
$$p \parallel x_{\text{test}} \quad \Rightarrow \quad t_{\text{test}} \cdot y_{\text{test}}$$

Strength: encourages decomposition into substeps.

Source: PAL paper, Section 2 and Figure 1.

Weakness: the LLM still has to execute every substep itself.

Where chain-of-thought can fail

CoT style

"The bakery baked 200 loaves. It sold 93 in the morning and 39 in the afternoon. $93 + 39 = 132$. 6 were returned. So $200 - 132 - 6 = 62$."

Issue

The verbal reasoning can look convincing, but a sign or arithmetic mistake changes the result.

Correct computation

```
loaves_baked = 200
sold_morning = 93
sold_afternoon = 39
returned = 6
answer = loaves_baked - sold_morning \
        - sold_afternoon + returned
# answer = 74
```

Transition to PAL

Let the LLM write the computation, but let an interpreter run it.

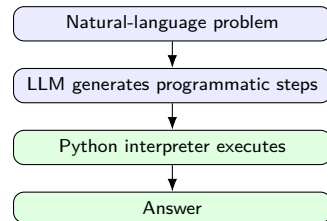
Source: Adapted from PAL paper, Figure 1.

Core idea of PAL

What changes?

PAL represents intermediate reasoning steps as a mixture of natural language and executable program statements.

- LLM: parse the problem and generate code.
- Python interpreter: execute the generated program.
- Final answer: obtained from the runtime result.



Key conceptual shift

The model no longer has to be a reliable calculator; it has to be a reliable translator from language to executable reasoning.

Source: PAL paper, Section 3.

Formal view of PAL prompting

Prompt format

In PAL, each in-context example contains the question x_i and a generated reasoning/program trace t_i , but the final answer does not need to be explicitly provided in the prompt because it will be obtained by execution.

$$p = \langle x_1 \cdot t_1 \rangle \parallel \langle x_2 \cdot t_2 \rangle \parallel \cdots \parallel \langle x_k \cdot t_k \rangle$$

$$p \parallel x_{\text{test}} \Rightarrow t_{\text{test}} \xrightarrow{\text{Python}} y_{\text{test}}$$

Important design choice

The paper stresses meaningful variable names, because they help the LLM ground code variables in the entities from the question.

Source: PAL paper, Section 3 and Section 6.

Example: mathematical reasoning with PAL

Question

Olivia has \$23. She bought five bagels for \$3 each. How much money does she have left?

```
money_initial = 23
bagels = 5
bagel_cost = 3
money_spent = bagels * bagel_cost
money_left = money_initial - money_spent
answer = money_left
```

- The LLM identifies quantities and operations.
- The answer is produced by executing Python.

Source: PAL paper, Figure 3.

Example: symbolic reasoning with PAL

Task type: colored objects

The model must track object order and attributes, then answer a relational question.

```
objects = [  
    ('paperclip', 'purple'),  
    ('stress ball', 'pink'),  
    ('keychain', 'brown'),  
    ('scrunchiephone charger', 'green')  
]  
  
stress_ball_idx = None  
for i, obj in enumerate(objects):  
    if obj[0] == 'stress ball':  
        stress_ball_idx = i  
        break  
  
direct_right = objects[stress_ball_idx + 1]  
answer = direct_right[1]
```

Source: Adapted from PAL paper, Figure 4.

Experimental setup

Task families

- Mathematical word problems
- Symbolic reasoning
- Algorithmic tasks

Baselines

- DIRECT prompting
- CoT prompting
- PAL prompting

Main backend

- Codex / code-davinci-002
- Greedy decoding, temperature 0
- Accuracy / solve rate

Why this setup matters

The experiments test whether the gain comes from program-aided execution rather than simply from using a larger language model.

Source: PAL paper, Section 4.

Benchmarks used in the paper

Category	Benchmarks / examples
Mathematical reasoning	GSM8K, GSM-HARD, SVAMP, ASDIV, SINGLEEQ, SINGLEOP, ADDSUB, MULTIARITH
Symbolic reasoning	COLORED OBJECTS, PENGUINS, DATE from BIG-Bench Hard
Algorithmic tasks	OBJECT COUNTING and REPEAT COPY

Presentation focus

I will focus on GSM8K/GSM-HARD for arithmetic robustness, and on COLORED OBJECTS/OBJECT COUNTING for symbolic and algorithmic reasoning.

Source: PAL paper, Sections 4.1–4.3.

Main math results: PAL improves across datasets

Method	GSM8K	GSM-HARD	SVAMP	MULTIARITH
DIRECT Codex	19.7	5.0	69.9	44.0
CoT Codex	65.6	23.1	74.8	95.9
CoT PaLM-540B	56.9	–	79.0	94.7
PAL Codex	72.0	61.2	79.4	99.2

Interpretation

The largest difference appears on GSM-HARD: when numbers become large, CoT accuracy drops sharply, while PAL remains much more stable because Python performs the arithmetic.

Source: PAL paper, Table 1 and Section 5.1.

Symbolic and algorithmic results

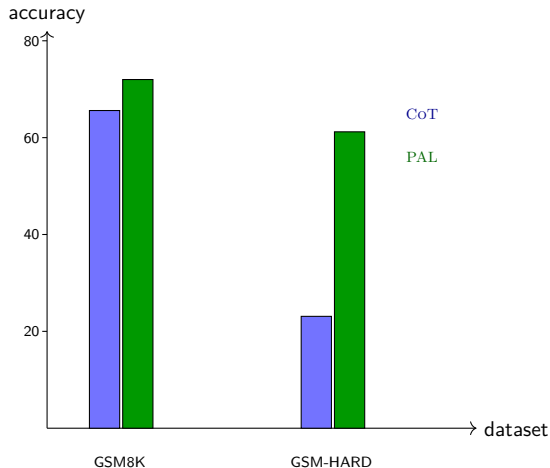
Method	Colored Objects	Penguins	Date	Repeat Copy	Objects Counting
DIRECT Codex	75.7	71.1	49.9	81.3	37.6
CoT Codex	86.3	79.2	64.8	68.8	73.0
PAL Codex	95.1	93.3	76.2	90.6	96.7

Interpretation

PAL is not only a calculator trick: it also helps when the task can be expressed as list operations, dictionaries, loops, filtering, or date computations.

Source: PAL paper, Table 2 and Section 5.2.

Robustness result: GSM8K vs. GSM-HARD



Source: PAL paper, Table 1 and Section 5.1.

GSM-HARD construction

The authors create a harder variant of GSM8K by replacing numbers in the questions with larger numbers, including values up to seven digits.

Observed failure mode

For many examples, CoT produces nearly the same reasoning text for the original and hard versions, suggesting that the main problem is arithmetic execution rather than decomposition.

Analysis: what actually makes PAL work?

Interpreter matters

When the model generated Python-like code but had to “execute” it itself, performance on GSM8K was far below full PAL.

Variable names matter

Replacing meaningful variable names with random names reduced accuracy; names help ground program variables to problem entities.

Not restricted to code-only LMs

The paper reports that sufficiently capable natural-language LMs can also benefit from PAL, although weaker models may struggle with code generation.

Runtime errors are rare

The generated Python code was syntactically correct, and fewer than 1% of examples raised runtime exceptions.

Source: PAL paper, Section 6.

Strengths and limitations

Strengths

- Clear neuro-symbolic division of labor.
- Strong gains on math, symbolic, and algorithmic tasks.
- Robustness to larger numbers.
- Simple implementation: prompt + interpreter.

Limitations / questions

- Correctness still depends on generating the right program.
- Execution safety and sandboxing are not the central focus.
- Some tasks may not map cleanly to Python programs.
- Evaluation is based on few-shot prompting with specific model families.

Open questions for the preliminary discussion

- 1 Should the final presentation emphasize **neuro-symbolic AI** more strongly, or stay closer to the prompting-method perspective?
- 2 Should I include a small live/demo example from my repository, or keep the seminar presentation fully paper-focused?
- 3 Which limitations are most relevant for the seminar: code execution safety, benchmark scope, or generalization beyond Python-solvable tasks?

Takeaway

PAL improves reasoning by making the LLM generate executable intermediate steps and delegating exact computation to a symbolic runtime.

- It keeps the strength of LLMs: natural-language understanding and decomposition.
- It avoids a key weakness of LLMs: unreliable arithmetic and step execution.
- It provides a simple but powerful neuro-symbolic pattern: **language model as planner, interpreter as executor**.

Source: PAL paper, Conclusion.

Thank you!

Questions and feedback